Prompting Methods Cheat Sheet

TechTown Start Studio | 15 Essential Methods for Non-Technical Founders

TIER 1 Core 5 | TIER 2 Next 10

1 MARKDOWN HEADERS

AI parses structure; forces YOU to organize

Bad
I need a landing page with hero and headline and CTA and features...

Good
Landing Page
Hero Section
- Headline: "Ship faster"

2 ALL CAPS FOR CONSTRAINTS

AI weights emphasized text more heavily

```
REQUIRED: Email MUST validate | NEVER allow empty fields |
ALWAYS show errors
```

Warning: Reserve for 2-4 critical items max.

3 CONSTRAINT SANDWICH

Context → Ask → Constraints works every time

```
## Context
E-commerce, React, 500+ products
## Request
Add search bar
## Constraints
MUST debounce, case-insensitive
```

4 INTERVIEW METHOD

Conversation beats mega-prompts

Risky
Build auth with login, OAuth, reset, sessions, profile...

Safer
Turn 1: "Start with login"
Turn 2: "Add OAuth"

5 REFERENCE + MODIFY

"Like X but Y" saves 1000 words

```
Build pricing page like Stripe's, but: 2 tiers, our colors,
add FAQ
```

6 NUMBERED STEPS

Removes ambiguity about order

```
1. Shipping form → 2. Order summary → 3. Payment → 4. Success
page
```

7 EXPLAIN BEFORE FIXING

Learn what went wrong, not just get a fix

```
[error] Before fixing: 1. Explain cause 2. Why in MY code? 3.
Then fix
```

8-9 SCREENSHOTS + FULL ERRORS

Visual > verbal; details matter

```
Screenshots: Mockups, competitor sites, error screens
Errors: Copy ENTIRE stack trace + what triggered it
```

10 CHUNKING

One feature per message = fewer errors

Too much
Add auth, dashboard, and dark mode

Chunked
Msg 1: auth | Msg 2: dashboard | Msg 3: dark mode

11 GIVE ME 3 OPTIONS

Forces alternatives, avoids tunnel vision

```
Give me 3 ways to structure DB. For each: schema, pros/cons,
when to use
```

12 COMPARISON MATRIX

Side-by-side evaluation reveals tradeoffs

```
Compare Supabase, Firebase, PlanetScale on: setup, pricing,
real-time, free tier
```

13 CRITIQUE YOURSELF FIRST

AI knows weaknesses but won't mention unless asked

```
Before implementing: What weaknesses? Edge cases? What would
senior eng push back on?
```

14 5 WHYS

Gets to root cause, prevents fixing wrong thing

```
Page slow → API 3s → Fetching 10k items → No pagination →
Fix: add pagination
```

15 PRE-MORTEM

"How could this fail?" overcomes optimism bias

```
Imagine it failed in 3 months. List 5 causes. What can we do
NOW to prevent each?
```

QUICK REFERENCE CARD

Method	When	Phrase
Markdown headers	Always	`# ## ###`
ALL CAPS	Critical constraints	`MUST NEVER`
Constraint sandwich	Non-trivial requests	Context→Ask→Limits
Interview method	Complex features	"Let's start with..."
Reference + Modify	UI/design work	"Like X but with..."

Method	When	Phrase
Numbered steps	Sequential processes	`1. 2. 3.`
Explain before fix	Debugging	"Why is this happening?"
Screenshots	UI work	[attach image]
Full error paste	Any error	Copy entire trace
Chunking	Multiple features	One per message

Method	When	Phrase
Give 3 options	Decisions	"Show 3 ways to..."
Comparison matrix	Evaluating choices	"Create a table..."
Critique yourself	Before building	"What's wrong with?"
5 Whys	Root cause	"Why? Why? Why?"
Pre-mortem	Risk planning	"How could this fail?"